

MLP + Parallel Mini-Batch SGD (MNIST)

TEAM: SINGLE THREADED

Ait-Ameur Mohamed Ilyas

Rayane Khelfa

Abada Abderahmane

Yanis Belharet

May 04, 2026

1. Introduction

1.1. Problem

The MNIST dataset is a subset of a larger set available from NIST. It is a somewhat classic example and often considered to be the “Hello, World!” of machine learning. It’s a ‘classification’ problem in the sense that we are expected to train a neural network to be able to recognize handwritten digits.

1.2. Dataset Description

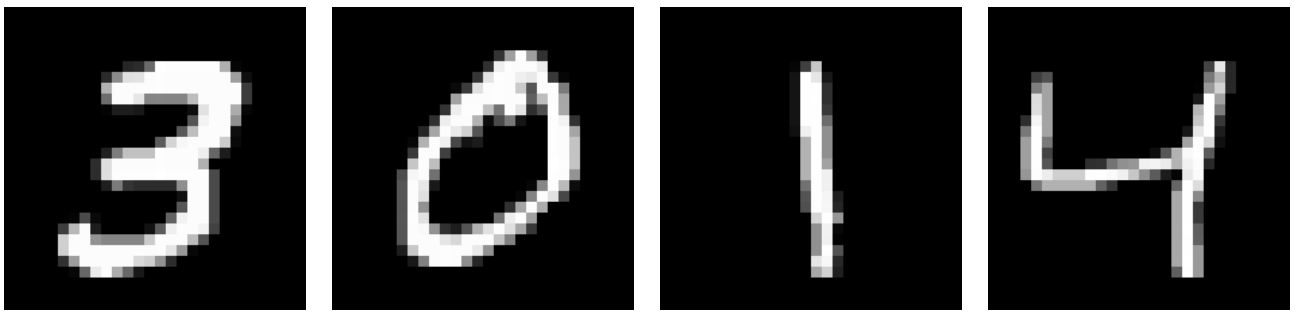


Figure 1: MNIST Dataset Samples

These are images with a very low resolution of 28×28 *greyscale* pixel values encoded on 8 bits (i.e. with the values of each pixel ranging from 0 to 255).

The dataset is split as follows:

```
dataset/MNIST/
├── t10k-images.idx3-ubyte  -> . Test images
├── t10k-labels.idx1-ubyte  -> . Test labels
├── train-images.idx3-ubyte  -> . Training images
└── train-labels.idx1-ubyte  -> . Training labels
```

Let’s look at the training files:

1.2.1. Image file (idx3):

```
[0..3]  magic number = 0x00000803 (from byte 0 to byte 3 inclusive)
[4..7]  num_images (60_000)
[8..11] num_rows   (28)
[12..15] num_cols  (28)
[16..]  pixels: uint8, row-major, one byte per pixel
```

1.2.2. Label file (idx1):

```
[0..3] magic number = 0x000000801
[4..7] num_labels (60_000)
[8..] labels: uint8, value 0-9
```

The magic number in each filetype serves as a sanity check since the dataset is written in BigEndian byte notation, it helps to make sure we are reading correctly. They also express the data type (0x08, unsigned byte) and the dimensionality (0x03, 60_000 2D images making this tensor 3D).

1.3. Motivation

Educating ourselves on neural networks and gradient descent learning algorithms as well as their parallelization.

2. Serial Implementation

2.1. Preprocessing

2.1.1. Data pipeline

I structured and read the files in a single contiguous memory block:

```
typedef struct Dataset {
    size_t size;
    Shape shape;
    uint8_t* labels;
    uint8_t* content;
    Image (*index)(Dataset* dataset, size_t index);
} Dataset;
```

the index() function gets a specific image out of the content field which is the entire dataset placed in a single contiguous memory block. The Image type is

```
typedef struct Image {
    Shape shape;
    uint8_t* lbl;
    uint8_t* content;
    void (*label)(Image img, uint8_t label);
    uint8_t (*get)(Image img, size_t row, size_t col);
    void (*set)(Image img, size_t row, size_t col, uint8_t value);

    void (*print)(Image img);
} Image;
```

which as it turns out is mostly just development utilities rather than useful algorithmic tools. The Shape is mostly as you can expect:

```
typedef struct Shape {
    size_t rows;
    size_t cols;
} Shape;
```

At first, I naively wrote:

```
for (int k = 0; k < size; k++) {
    Image img = dataset->index(dataset, k);
    img.label(img, readbyte(fl));
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            img.set(img, i, j, readbyte(fi));
        }
    }
}
```

Yeah. Per-byte file reads. `readbyte()` is called $60000 \times 784 \approx 47$ M times. I was paying 47 M function calls. The whole image block is contiguous on disk. it can be read in one shot:

```
fread(dataset->content, 1, size * rows * cols, fi);
fread(dataset->labels, 1, size, fl);
```

That replaces the entire for k / for i / for j loop and is dramatically faster.

normalize to float in [0,1]. One-hot the labels (or keep as `uint8_t` and index into the loss).

2.2. Algorithm Description & Complexity Analysis

2.2.1. Network

- **Architecture:** $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ with ReLU on hidden layers and softmax on the output.
- Store each layer as `W[out][in]`, `b[out]`, plus buffers for activations and gradients. Allocate once, reuse across iterations.
- Init: He/Xavier for weights, zeros for biases.

2.2.2. Forward / backward / update

- **Forward:** matrix-vector (or matrix-matrix once you batch) + activation per layer.
- **Loss:** cross-entropy with softmax — fuse them so the gradient at the output is just $\text{softmax}(z) - y_{\text{onehot}}$.
- **Backward:** chain rule layer by layer; each layer needs its input activation (saved during forward) to compute dW .
- **Update:** vanilla SGD

$$W - = \text{lr} * \frac{dW}{\text{batch_size}}$$

2.2.3. Training loop

- Shuffle indices each epoch, iterate mini-batches (e.g. 64 or 128), forward $\rightarrow$ backward $\rightarrow$ update.
- Track loss + accuracy on the test set per epoch.

3. Parallel Implementation

3.1. Preprocessing

3.2. Design Rational

3.3. Data/Task Decomposition

3.4. Synchronisation

4. Experimental Results

4.1. Timing Table

4.2. Speedp Graph

4.3. Accuracy Comparison

5. Conclusion and Perspectives

5.1. Findings

5.2. Limitations

5.3. Future Direction

6. References